VeriNC

Finding Design Risks of In-Network Computing Systems

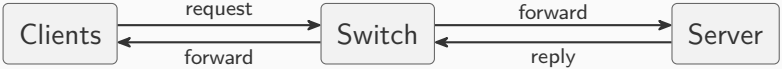
Tianyu Bai[1] Xiaoxi Zhang[2] Haoqing Wang[1] Ying Zhang[3] Wenfei Wu[1]

[1]Peking University · [2]Sun Yat-sen University · [3]Meta

ICNP 2026 · Tempe, AZ

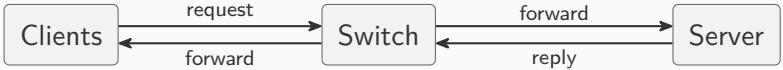
Computation Is Moving Into the Switch

Traditional Networks



Computation Is Moving Into the Switch

Traditional Networks

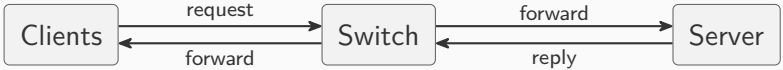


In-Network Computing (INC)



Computation Is Moving Into the Switch

Traditional Networks



In-Network Computing (INC)



Tensor aggregation (SwitchML) · Key-value cache (NetCache) · Lock manager (NetLock)

Why Correctness Is Hard Here

Network unreliability leads to computational errors.

TCP makes delivery reliable *between the endpoints*. The switch in the middle still sees loss, duplication and reordering.

Why Correctness Is Hard Here

Network unreliability leads to computational errors.
TCP makes delivery reliable *between the endpoints*. The switch in the middle still sees loss, duplication and reordering.

And the switch cannot recover from them.
No loops. One memory access per pipeline stage. No reliable transport in the data plane.

Why Correctness Is Hard Here

Network unreliability leads to computational errors.
TCP makes delivery reliable *between the endpoints*. The switch in the middle still sees loss, duplication and reordering.

And the switch cannot recover from them.
No loops. One memory access per pipeline stage. No reliable transport in the data plane.

This is what distinguishes INC from a traditional distributed system:
there, every node is a general-purpose server
with rich computation and storage resources.

NetCache Returns a Value That Was Overwritten

Client

writes, then reads back

Switch

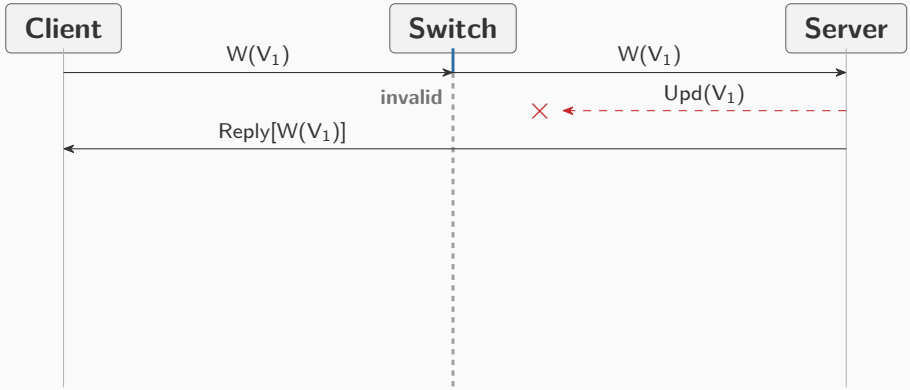
caches the item, answers reads itself

Server

source of truth, keeps it in sync

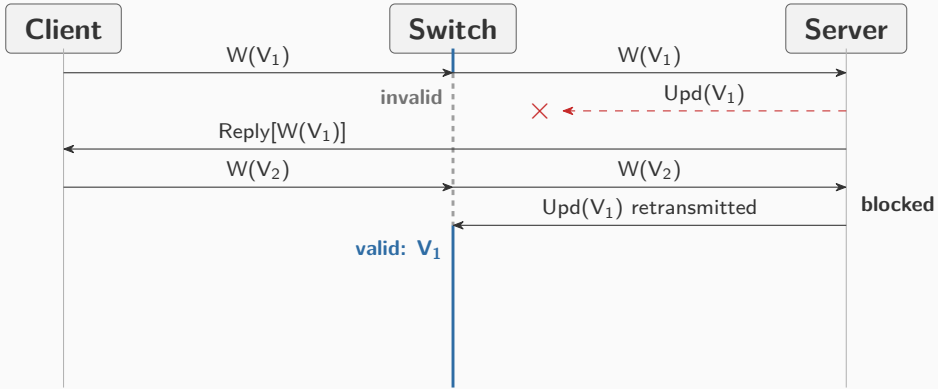
A write-through cache. Watch what the switch believes about it.

NetCache Returns a Value That Was Overwritten



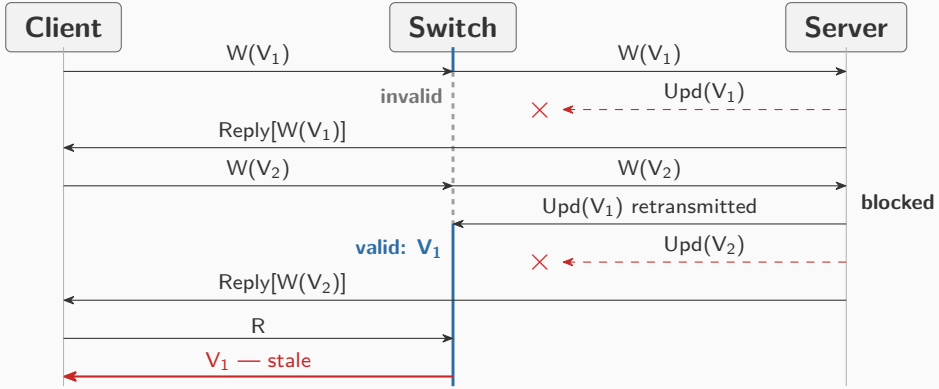
The client is told the write succeeded — but the cache never heard about it.

NetCache Returns a Value That Was Overwritten



The retransmitted *old* update marks the cache valid again.

NetCache Returns a Value That Was Overwritten



Cache consistency is violated: the read is not concurrent with any write, but does not get the latest value.

This Is Not Something You Find by Thinking Harder

Two specific losses, in one specific interleaving.

Protocols get designed for a reliable network, then *patched* for the exceptions. That fails three ways.

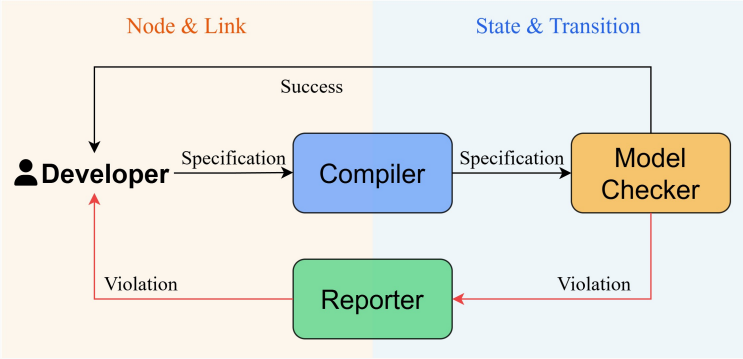
- The exceptions you never thought of are never patched
- A patch addresses one specific exception, not the root cause
- Patches conflict with each other once several exceptions co-occur

Synthetic workloads and simulators do not exhaustively enumerate these.

Existing Verification Does Not Answer This

- **Network verification** checks routing — reachability, loops, isolation. No computation involved.
- **HSA and NetSMC** assume a stateless network, or reason about one packet at a time.
  INC switches hold state across many related packets.
- **P4 program verifiers** (p4v, Vera, Aquila) check the *implementation*. We ask whether the *design* itself is correct.

VeriNC



VeriNC explores all possible execution traces — exhaustively, within the bounds you set.

The Abstraction You Work In

- A **node** is a machine, running several **threads** — application, receive, retransmission.
  Threads run concurrently; every interleaving gets explored.

The Abstraction You Work In

- A **node** is a machine, running several **threads** — application, receive, retransmission.
  Threads run concurrently; every interleaving gets explored.
- Nodes are joined by **links**. Three primitives: `Send`, `Receive`, `Multicast`.

The Abstraction You Work In

- A **node** is a machine, running several **threads** — application, receive, retransmission.
  Threads run concurrently; every interleaving gets explored.
- Nodes are joined by **links**. Three primitives: `Send`, `Receive`, `Multicast`.
- The network is **reliable**, **lossy**, or **out-of-order** — set globally, or per link.

The Abstraction You Work In

- A **node** is a machine, running several **threads** — application, receive, retransmission.
  Threads run concurrently; every interleaving gets explored.
- Nodes are joined by **links**. Three primitives: `Send`, `Receive`, `Multicast`.
- The network is **reliable**, **lossy**, or **out-of-order** — set globally, or per link.

Users specify protocol behavior
without modeling the network itself.

What You Actually Write

```
configuration {
  MAX_LOSS = 1;  CLIENT_NUM = 3;  REQ_NUM = 5;  ...
}
topology {
  nodetype Client, Switch, Controller, Server;
  link c1, c2, c3 -- sw -- ctrl, s;  ...
}
protocol {
  var(Switch) cached = 0, valid = 0, value = null;  ...
  thread(Switch) SwitchRecv { ... }
}
```

Protocol logic, not P4 — the core logic, without implementation detail.

Saying What Correctness Means

- An **assert statement** — a first-order logic formula, checked where it sits
- A **property** block in LTL, checked after every transition

  [] always <> eventually
- A **stateful check function**, for complex temporal properties

Challenge: The State Space Explodes

Every packet may be dropped, reordered, or retransmitted at any moment.

Challenge: The State Space Explodes

Every packet may be dropped, reordered, or retransmitted at any moment.

- **INC objects are usually logically independent.**
  So we model just one of them.

Challenge: The State Space Explodes

Every packet may be dropped, reordered, or retransmitted at any moment.

- **INC objects are usually logically independent.**
  So we model just one of them.
- **Nodes of the same type are symmetric.**
  So we explore one arrangement and discard its permutations — without losing completeness.

Challenge: The State Space Explodes

Every packet may be dropped, reordered, or retransmitted at any moment.

- **INC objects are usually logically independent.**
  So we model just one of them.
- **Nodes of the same type are symmetric.**
  So we explore one arrangement and discard its permutations — without losing completeness.
- **A little unreliability is usually enough.**
  So we constrain it: a maximum number of losses, reorderings and duplicates.

What It Costs to Run

System	VeriNC	TLA$^+$	To first violation
FissLock	466	1053	1 s
SwitchML	98	440	3 s
NetCache	190	589	8 s
NetReduce	146	486	72 s

On average the specification is **67.2% shorter** than the TLA$^+$ it compiles to.

Violations usually sit in the *shallow* layers of the state space, so breadth-first search reaches them early.

Seven of Nine Systems Violate Properties

System	Property violated	Network
SwitchML	computational correctness	out-of-order
ATP	memory leak freedom	out-of-order
NetReduce	terminality	lossy
NetCache	cache consistency	lossy
NetCache	terminality	reliable
FarReach	cache consistency	reliable
NetLock	lock exclusion	delayed
FissLock	lock exclusion	delayed

Seven of Nine Systems Violate Properties

System	Property violated	Network
SwitchML	computational correctness	out-of-order
ATP	memory leak freedom	out-of-order
NetReduce	terminality	lossy
NetCache	cache consistency	lossy
NetCache	terminality	reliable
FarReach	cache consistency	reliable
NetLock	lock exclusion	delayed
FissLock	lock exclusion	delayed

Few of these systems explicitly state their network assumptions.

Not Just on Paper

- Following each violation trace, we injected exactly those losses and delays into the **real open-source implementations**.

Not Just on Paper

- Following each violation trace, we injected exactly those losses and delays into the **real open-source implementations**.
- **All six open-source systems reproduced the violation.**

Not Just on Paper

- Following each violation trace, we injected exactly those losses and delays into the **real open-source implementations**.
- **All six open-source systems reproduced the violation.**
- The authors of **ATP** and **NetReduce** confirmed these were oversights at design time.

Not Just on Paper

- Following each violation trace, we injected exactly those losses and delays into the **real open-source implementations**.
- **All six open-source systems reproduced the violation.**
- The authors of **ATP** and **NetReduce** confirmed these were oversights at design time.
- **EPIC** (SIGCOMM '26) used VeriNC *during* its design — a buffer management pitfall was fixed before publication.

Three Patterns Behind Those Violations

- **Stale packets pollute switch state.**
  Give the switch a way to recognize them.
- **Concurrent events race with no coordination.**
  Handle how events interact, not one event at a time.
- **Unreliability turns a latent risk into a real violation.**
  Most of what we found needs only a single such event.

Conclusion

- **Design-level verification** for in-network computing systems
- A high-level **specification language** and **configurable network environments**
- Nine systems modeled; design risks in seven of them, found in seconds and reproduced on the real implementations

Formal modeling and verification are an essential step in driving the deployment of INC systems.

Thank you

Questions?

github.com/cs-icez/verinc-compiler

github.com/cs-icez/verinc-violation

tianyubai@stu.pku.edu.cn